

AYRIK MATEMATİK (MTM2622)

10.06.2021 - 12. DERS

İletişim Bilgileri

Arş. Gör. Dr. Fatih Aylıkçı

Kimya-Metalurji Fakültesi

Matematik Mühendisliği A237

02123834616 - faylikci@yildiz.edu.tr

Avesis: <https://avesis.yildiz.edu.tr/faylikci>

Konular

1. Sayılar Teorisi ve Şifreleme
2. Modüler Aritmetik, Bölünebilirlik, Asal sayılar ve EBOB
3. Kongrüans çözümleri ve uygulamaları
4. Boolean cebrine giriş , Boolean fonksiyonları
5. Boolean temsilleri, Devrelerin minimizasyonu
6. Rekürans bağıntıları ve uygulamaları
7. Graf (çizge) ve graf (çizge) modelleri, Graf terminolojisi ve Özel tipten graflar
8. Euler ve Hamilton Yolları, En Kısa Yol Problemleri
9. Planar Graflar
10. Graflarda Renklendirme
11. Ağaçlar ve uygulamaları
12. Temel Ağaçlar, Minimum Temel Ağaçlar
13. Minimum Temel Ağaçlar ile ilgili uygulamalar

Kaynak Kitaplar

1. Kenneth H. Rosen, Discrete Mathematics and Its Applications, 7th Edition McGraw-Hill Companies, Inc., 2012.
2. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, Pearson Addison Wesley, 2004.
3. Kevin Ferland, Discrete Mathematics: An Introduction to Proofs and Combinatorics, 1st Edition, Cengage Learning, 2008.
4. Richard Johnsonbaugh, Discrete Mathematics, 7th Edition, Pearson Education, 2013.

Değerlendirme

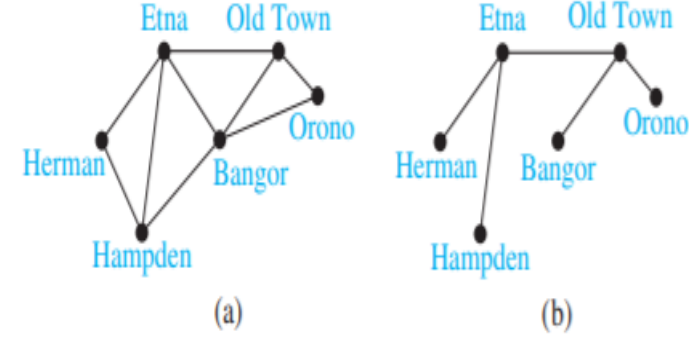
%30 1. Vize

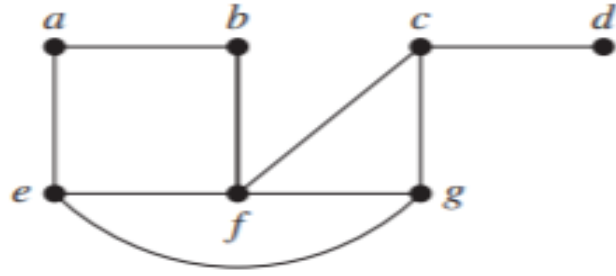
%30 2. Vize (Sınav, Ödev veya Quiz)

%40 Final

Kapsayan Ağaçlar

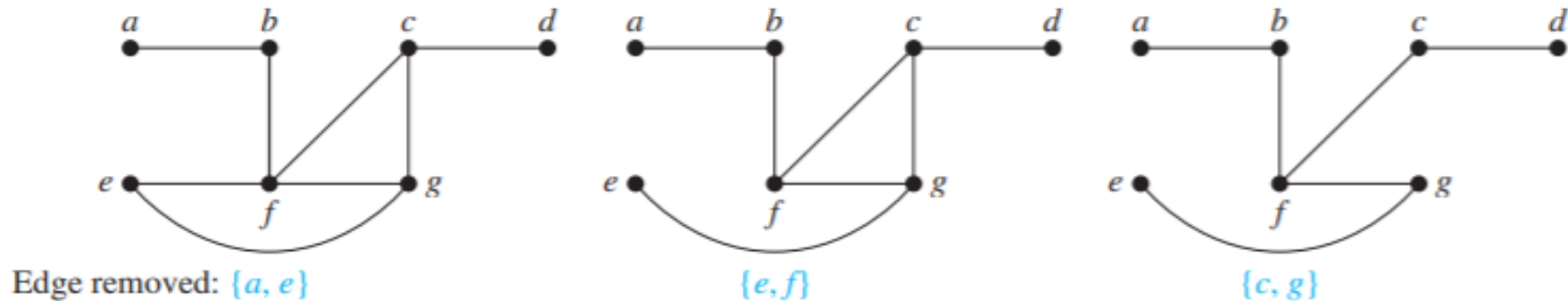
- Şekil (a)'da gösterilen basit çizge ile temsil edilen yol sistemini ele alalım. Kışın yolları açık tutmanın tek yolu yollardaki karı düzenli olarak temizlemektir. Karayolları müdürlüğü, en az sayıdaki yolu temizleyerek herhangi iki ilçenin arasının her zaman açık olmasını sağlamak istemektedir. Bu nasıl yapılabilir?
- Herhangi iki ilçe arasında bir geçiş olması için en az beş yol temizlenmelidir. Şekil (b) böyle bir yol kümesini göstermektedir. Bu yolları gösteren alt çizge bir ağaçtır, çünkü bağlantılıdır ve altı düğümü ve beş kenarı vardır.
- Bu problem, verilen basit çizgenin tüm düğümlerini kapsayan en az kenara sahip bir bağlantılı alt ağaç ile çözüldü. Böyle bir çizge ağaç olmalıdır.
- **TANIM 1:** G basit bir çizge olsun. G 'nin kapsayan ağacı, G 'nin bir alt çizgesi olan ve G 'nin her düğümünü kapsayan bir ağaçtır.
- Bir kapsayan ağacına sahip bir basit çizge bağlantılı olmalıdır, çünkü kapsayan ağacında herhangi iki düğüm arasında bir yol vardır. Bunun tersi de doğrudur, yani her basit çizgenin bir kapsayan ağacı vardır. Bu sonucu ispat etmeden önce bir örnek vereceğiz.



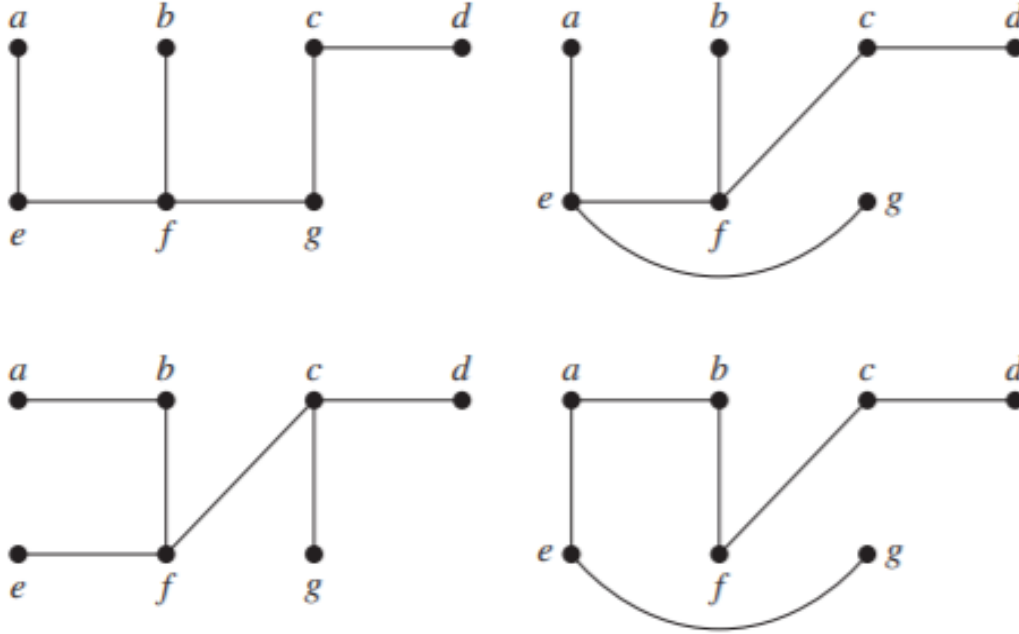


- Örnek 1 Üstteki şekilde gösterilen G basit çizgesinin bir kapsayan ağacını bulunuz.

G çizgesi bağlantılıdır, fakat bir ağaç değildir çünkü basit devreler içerir. $\{a,e\}$ kenarını çıkarınız. Bu bir basit devreyi eler ve ortaya çıkan alt ağaç hala bağlantılı ve hala G'nin tüm düğümlerini içerir. Sonra ikinci basit devreyi elemek için $\{e,f\}$ kenarını çıkarınız. Son olarak da basit devre içermeyen bir basit çizge elde etmek için $\{c,g\}$ kenarını çıkarınız. Bu alt ağaç bir kapsayan ağaçtır, çünkü G'nin her düğümünü kapsayan bir ağaçtır. Kapsayan ağacı elde etmek için çıkarılış sırası alttaki şekilde gösterilmiştir.



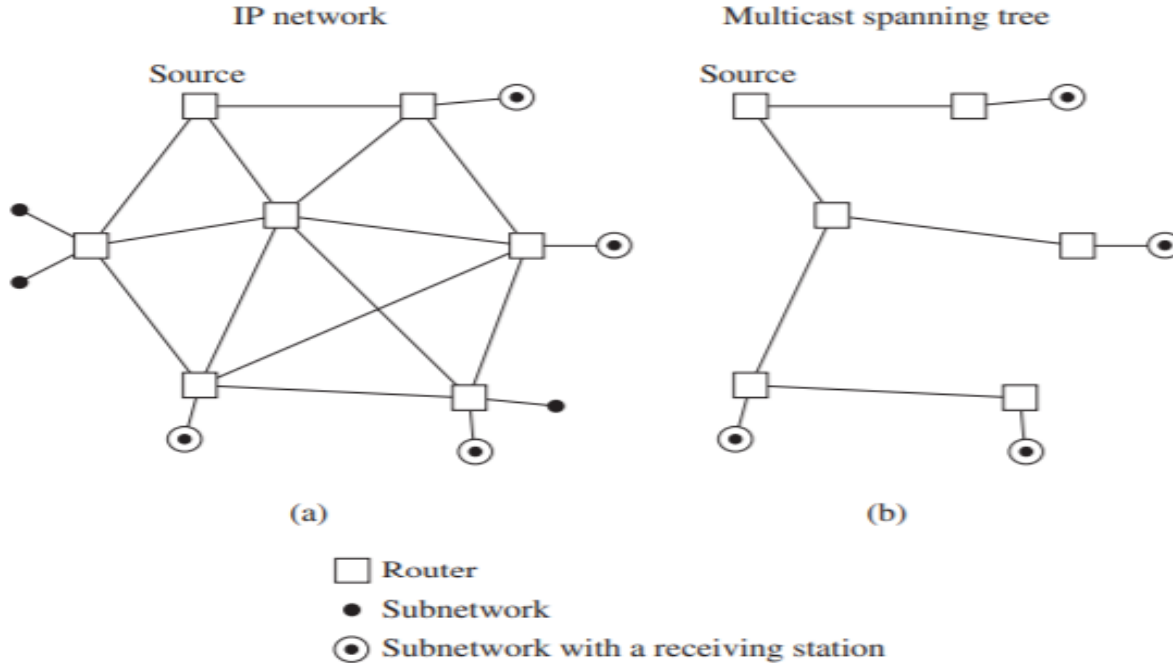
- G'nin tek kapsayan ağacı bir önceki slaytın altındaki şekildeki ağaç değildir. Örneğin aşağıdaki şekilde gösterilen ağaçların her biri G'nin kapsayan bir ağacıdır.



- **TEOREM 1:** Basit bir çizge ancak ve ancak bir kapsayan ağaca sahip olduğunda bağlantılıdır.
- **İspat:** İlk olarak bir G basit çizgesinin bir T kapsayan ağacı olsun. T , G 'nin her düğümünü içerir. Ayrıca, düğümlerin herhangi ikisi arasında T 'nin içinde bir yol vardır. T , G 'nin bir alt ağacı olduğu için, düğümlerin herhangi ikisi arasında G 'nin içinde bir yol vardır. Dolayısıyla G bağlantılıdır.
- Şimdi G bağlantılı olsun. Eğer G bir ağaç değilse, bir basit devre içermelidir. Bu basit devrelerden birinden bir kenar çıkarın. Ortaya çıkan alt ağacın bir kenarı eksiktir fakat hala G 'nin tüm düğümlerini içerir ve bağlantılıdır. Bu alt ağaç hala bağlantılıdır, çünkü çıkarılan kenarı içeren bir yol ile bağlı olan iki düğüm, bu kenarı içermeyen bir yol ile de bağlıdır. Böyle bir yolu, orijinal yola çıkarılan kenarın bulunduğu noktada bu kenarı çıkarılan basit devreyi ekleyerek elde ederiz. Eğer bu alt çizge ağaç değilse, bir basit devresi vardır, daha önceki gibi bir basit devre içindeki bir kenarı çıkarınız. Bu süreci hiçbir basit devre kalmayana kadar devam ettiriniz. Bu mümkündür, çünkü çizgede sadece sonlu sayıda kenar vardır. Süreç basit devre kalmadığında sona erer. Bir ağaç oluşturulur, çünkü kenarlar çıkarıldığında çizge bağlantılı kalmıştır. Bu bir kapsayan ağaçtır, çünkü G 'nin tüm düğümlerini içerir.

- Kapsayan ağlar aşağıdaki örnekte gösterildiği gibi veri ağlarında önemlidir.
- Örnek 2: (IP Çoklu Yayın) Kapsayan ağlar Internet Protokol (IP) ağları üzerinde yapılan çoklu yayınlamada önemli bir rol oynar. Bir ana bilgisayardan her biri bir alt ağ olan çoklu alıcı bilgisayarlara veri yollamak için, veri her bilgisayara ayrı ayrı yollanabilir. Tekli yayın denen bu tür iletişim, verimsizdir, çünkü ağ üzerinden aynı verinin pek çok kopyası iletilir. Çoklu alıcı bilgisayarlara veri iletiminin daha verimli olması için IP çoklu yayını kullanılır. IP çoklu yayın ile bir bilgisayar verinin bir kopyasını ağa yollar ve veri ara yönlendiricilere ulaştığında veri bir veya daha fazla yönlendiriciye iletilir. Bunun sonucunda çeşitli alt ağlarda bulunan tüm alıcı bilgisayarlar bu veriyi alır. (Yönlendiriciler, bir ağ içindeki alt ağlar arasında IP datagramları iletmeye tahsis edilmiş bilgisayarlardır. Çoklu yayında, yönlendiriciler her biri alıcı bilgisayarların katılabileceği bir seansı temsil eden Class D adresleri kullanırlar.)

- Verinin alıcı bilgisayarlara mümkün olduğunca hızla ulaşması için verinin ağ içerisinde aldığı yolda döngülerin (çizge teorisi terminolojisinde devre veya döngü) olmaması gerekir. Yani, veri belli bir yönlendiriciye ulaştığında, bu yönlendiriciye geri dönmemelidir. Döngülerden kaçınmak için çoklu yayın yönlendiricileri, düğümleri çoklu yayın kaynağı, yönlendiriciler ve alıcı bilgisayarları içeren alt ağlar; kenarları bilgisayarlar ve/veya yönlendiriciler arasındaki bağlantılar olan çizgede bir kapsayan ağaç oluşturmak için ağ algoritmaları kullanır. Bu kapsayan ağacın kökü çoklu yayın kaynağıdır. Alıcı bilgisayarları içeren alt ağlar ağacın yapraklarıdır. (Alıcı istasyonları içermeyen alt ağlar çizgeye alınmamıştır.) Bu, şekilde gösterilmiştir.

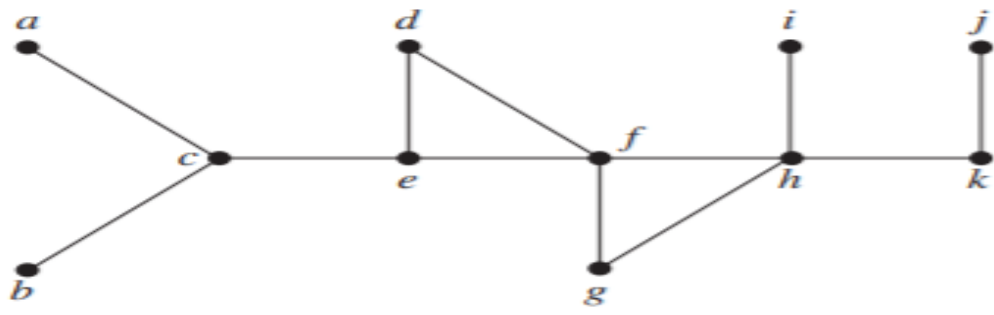


Derinlik-Öncelikli Arama

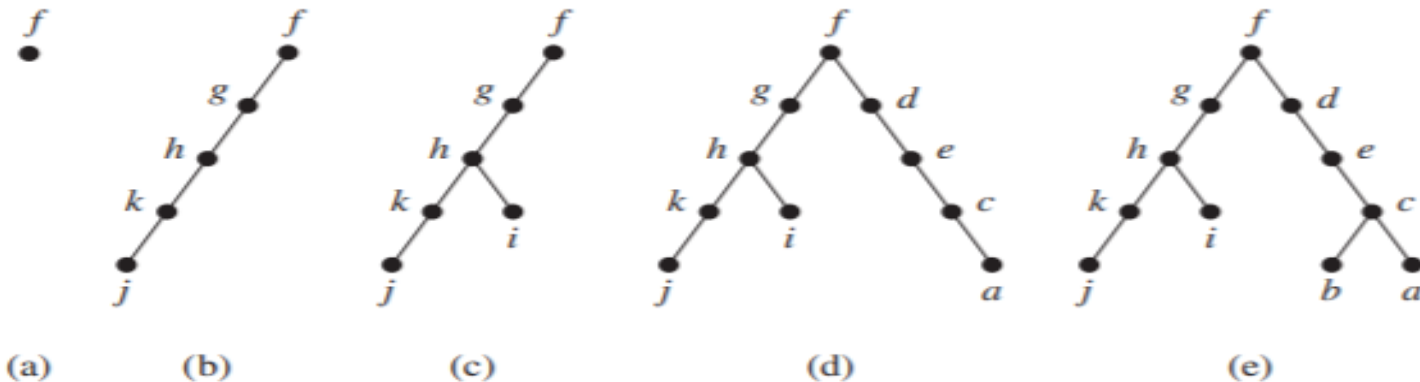
- Teorem 1'in ispatı, basit devrelerden kenarlar çıkararak kapsayan ağacı bulan bir algoritma verir. Bu algoritma verimsizdir, çünkü bu basit devrelerin belirlenmesini gerektirir. Kapsayan ağaçları kenar çıkararak oluşturmak yerine, kapsayan ağaçları ardarda kenar ekleyerek inşa edebiliriz. Bu prensibe dayalı iki algoritma burada sunulacaktır.
- Bir bağlantılı basit çizge için bir kapsayan ağacı **Derinlik-Öncelikli Arama** kullanarak inşa edebiliriz. Bir köklü ağaç oluşturacağız ve kapsayan ağacı da bu köklü ağacın temel yönsüz çizgesi olacaktır. **Bu çizgenin herhangi bir düğümünü kök olarak seçiniz.** Bu düğümden başlayarak, her yeni kenar yoldaki son düğüm ile alakalı olacak şekilde kenarlar ve bir düğüm önceden yolda olmayacak şekilde düğümler ekleyerek bir yol oluşturunuz. Bu şekilde yola düğümleri ve kenarları eklenebildiği kadar ekleyiniz. **Eğer yol çizgenin tüm düğümlerinden geçerse, bu yoldan oluşan ağaç bir kapsayan ağaçtır.** Ancak, yol tüm düğümlerden geçmezse daha fazla düğümler ve kenarlar eklenmelidir. Yoldaki son düğümün yanına geri dönünüz ve eğer mümkünse bu düğümden başlayarak önceden ziyaret edilmeyen düğümlerden geçen yeni bir yol oluşturunuz. Eğer bu yapılmazsa, yolda bir düğüm daha geri dönünüz, yani yoldaki iki düğüm öncesine dönünüz ve tekrar deneyiniz.

- Bu prosedüre devam ediniz, son ziyaret edilen düğümden başlayarak, bir anda bir düğüm geri dönerek daha fazla kenar eklenmesi mümkün olmayana kadar yeni yollar oluşturunuz. Çizge sonlu sayıda kenara sahip ve bağlantılı olması sebebiyle bu prosedür bir kapsayan ağaç ortaya çıkmasıyla biter. Algoritmanın bir aşamasındaki bir yolu bitiren her düğüm köklü ağaçta bir yaprak olacaktır ve kendisinden başlanılarak bir yol oluşturulan her düğüm bir iç düğüm olacaktır.

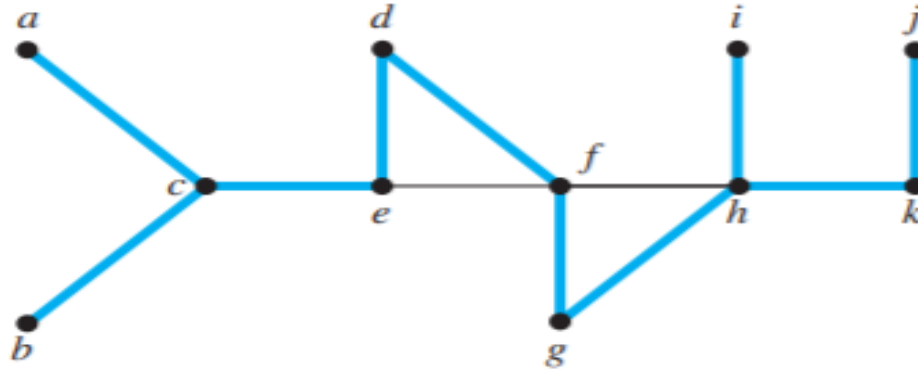
- Çizgedeki düğümler sıralıysa, prosedürün her aşamasındaki kenarların seçimi, sıralamadaki müsait ilk düğümü seçtiğimizde belirlenir. Ancak, bir çizgedeki düğümleri her zaman tam olarak sıralamayacağız.
- Derinlik-Öncelikli Aramaya **geri-iz sürme** de denir, çünkü algoritma yollar eklemek için önceden ziyaret edilen düğümlere geri döner.



- Örnek 3: Derinlik-Öncelikli Aramayı kullanarak üstteki şekilde gösterilen G çizgesinin bir kapsayan ağacını bulunuz.
- G'nin bir kapsayan ağacını ortaya çıkaran Derinlik-Öncelikli Aramanın kullandığı adımlar alttaki şekilde gösterilmiştir. Rastgele bir f düğümü ile başlarız. Bir yol, önceden yolda olmayan düğümlerle alakalı kenarları mümkün olduğu sürece ardı ardına ekleyerek oluşturulur. Bu f,g,h,k,j yolunu oluşturur (başka yollar da oluşturulabilirdi). Sonra k'ya geri iz sürünüz. k'da başlayan daha önceden ziyaret edilmeyen düğümleri içeren bir yol yoktur. Dolayısıyla h'ye geri iz süreriz. h,i yolunu oluşturunuz. Sonra g'ye ve sonra da f'ye geri iz sürünüz. f'den başlayarak f,d,e,c,a yolunu oluşturunuz. Sonra c'ye geri iz sürünüz ve c,b yolunu oluşturunuz. Bu kapsayan ağacı ortaya çıkarır.



- Bir çizgenin Derinlik-Öncelikli Arama ile seçilen kenarlarına ağaç kenarları denir. Çizgenin diğer tüm kenarları bir düğümü, bu düğümün bir atasına veya nesline bağlamalıdır. Bu kenarlara arka kenarlar denir.
- Örnek 4: Şekilde f düğümünden başlayarak yapılan Derinlik-Öncelikli Arama ile bulunan ağaç kenarlarını kalın renkli doğrular ile vurgulamaktayız. (e,f) ve (f,h) kenarları ince siyah doğrular ile gösterilmiştir.



Genişlik-Öncelikli Arama

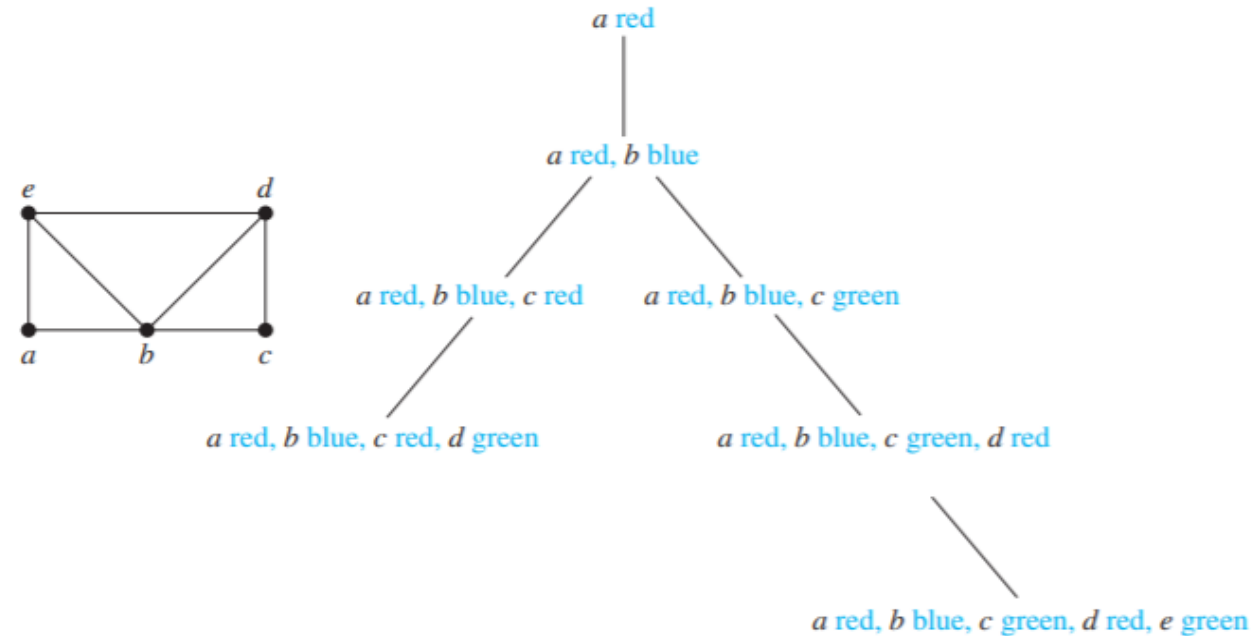
- Bir basit ağaç için bir kapsayan ağacı **Genişlik-Öncelikli Arama** ile de oluşturabiliriz. Yine bir köklü ağaç oluşturulacaktır ve bu köklü ağacın temel yönsüz ağacı kapsayan ağacı oluşturacaktır. **Çizgenin düğümlerinden rastgele bir kök seçiniz.** Sonra bu düğüm ile alakalı tüm kenarları ekleyiniz. Bu aşamada eklenen yeni düğümler kapsayan ağacın seviye 1'deki düğümleri olur. Bunları rastgele sıralayınız. Daha sonra, seviye 1'deki sırayla ziyaret edilen her düğüm için, bu düğümle alakalı her kenarı bir basit devre oluşturmadığı sürece ağaca ekleyiniz. Seviye 1'deki her düğümün çocuğunu rastgele sıralayınız. Bu ağacın seviye 2'deki düğümlerini oluşturur. Ağacın tüm düğümleri eklenene kadar bu prosedüre devam ediniz. Çizgenin sadece sonlu sayıda kenarı olduğu için bu prosedür eninde sonunda bitecektir. Çizgenin her düğümünü içeren bir ağaç oluşturduğunuzdan dolayı bir kapsayan ağaç elde ederiz.

Geri-İz Sürme Uygulamaları

- Sadece tüm olası çözümlerin tam kapsamlı araması yapılarak çözülebilecek problemler vardır. Çözüm için sistemli arama yapmanın bir yolu, her iç düğümü bir kararı ve her yaprağı olası bir çözümü temsil eden bir karar ağacı kullanmaktır. Geri iz sürme ile bir çözüm bulmak için, ilk olarak mümkün olduğu sürece bir çözüme ulaşma girişiminde bulunacak bir kararlar dizisi oluşturunuz. Bu kararlar dizisi, karar ağacında bir yol ile temsil edilebilir. Kararlar dizisini büyüterek bir sonuç elde edilemeyeceği bilindiğinde, bulunulan düğümün ebeveynine geri iz sürünüz ve eğer mümkünse çözüme gidecek başka bir kararlar dizisi üzerine çalışınız. Bu prosedür bir çözüm bulunana kadar devam eder veya çözüm olmadığı ortaya çıkar.

- Örnek 6: Çizge Renklendirmesi (Boyama) Bir çizgenin n renk ile boyanıp boyanamayacağına karar vermek için geri iz sürme nasıl kullanılabilir?
- Bu problemi geri iz sürme kullanarak şöyle çözebiliriz. İlk önce bir a düğümü seçiniz ve renk 1'i atayınız. Sonra, ikinci bir b düğümü seçiniz ve eğer b , a 'nın komşusu değilse renk 1'i atayınız. Aksi takdirde b 'ye renk 2'yi atayınız. Daha sonra üçüncü düğüm olan c 'ye gidiniz. Eğer mümkünse c için renk 1'i kullanınız. Aksi takdirde, eğer mümkünse renk 2'yi kullanınız. Sadece ne renk 1 veya ne de renk 2 kullanılabildiğinde renk 3 kullanılmalıdır. Her zaman listedeki ilk izin verilen renk kullanılarak her eklenen düğüme n renkten birini atama mümkün olduğu sürece bu işleme devam ediniz. Eğer n renkten herhangi biri ile boyanamayan bir düğüme ulaşırsa, son atamaya geri iz sürünüz ve son boyanan düğümü eğer mümkünse listedeki izin verilen bir sonraki renk ile değiştiriniz. Eğer bu boyamayı değiştirmek mümkün değilse, bir düğümün rengini değiştirmek mümkün olana kadar bir anda bir adım geri giderek daha önceki atamalara geri iz sürünüz. Mümkün olduğu sürece, eklenen düğümlere renk ataması yapmaya devam ediniz. Eğer n rengi kullanarak bir boyama varsa geri iz sürme bunu oluşturacaktır. (Ne yazık ki bu prosedür oldukça verimsizdir.)

- Şekilde gösterilen çizgenin renk boyama problemini ele alalım. Şekilde gösterilen ağaç, iz sürmenin bir 3-renk boyamanın oluşturulmasında nasıl kullanıldığını göstermektedir. Bu prosedürde, ilk önce kırmızı kullanılır, sonra mavi ve en son da yeşil. Bu basit örneğin geri iz sürmesiz de yapılabileceği aşikardır, fakat tekniğin güzel bir anlatımıdır.
- Bu ağaçta a'ya kırmızıyı atayan kökten çıkan başlangıç yolu, a'yı kırmızı, b'yi mavi, c'yi kırmızı ve d'yi yeşil yapan boyamaya götürür. a,b,c ve d bu şekilde boyandığında e'yi üç renkten herhangi biri ile boyamak mümkün değildir. Böylece, bu boyamayı temsil eden düğümün atasına geri iz sürünüz. d için başka bir renk kullanılamayacağı için bir seviye daha geriye iz sürünüz. Sonra c'nin rengini yeşile çeviriniz. d'ye kırmızıyı ve e'ye yeşili atayarak çizgenin bir boyanmasını elde ederiz.

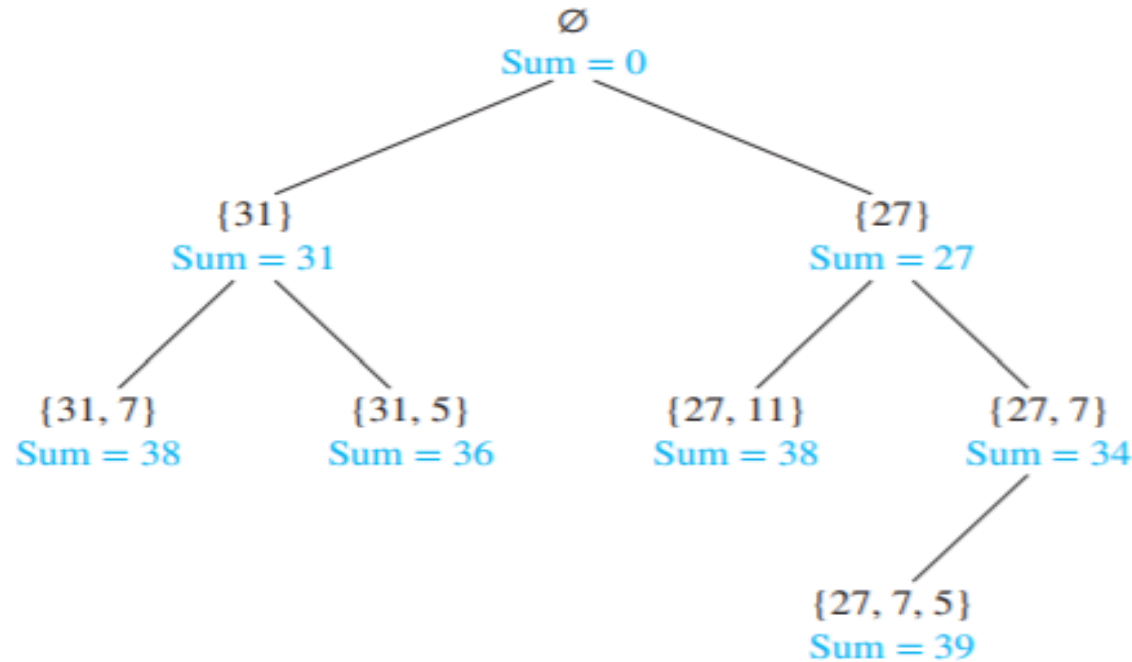


- Örnek 7: n-Vezir Problemi n-vezir problemi, herhangi iki vezir birbirine hamle yapamayacak şekilde n veziri $n \times n$ 'lik satranç tahtasına nasıl yerleştirilebileceğini sorar. Geri iz sürme, n-vezir problemini çözmek için nasıl kullanılabilir?
- Bu problemi çözmek için $n \times n$ 'lik bir satranç tahtasında herhangi iki konumu aynı satırda, aynı sütunda veya aynı köşegende [bir köşegen, bir m için $i+j = m$ veya $i-j = m$ olan tüm (i,j) konumlarından oluşur] olmayan n konum bulmalıyız. n-vezir problemini çözmek için geri iz sürmeyi kullanacağız. Boş bir satranç tahtası ile başlayacağız. $(k+1)$ inci aşamada ilk k sütunda vezirler olacağı için $(k+1)$ inci sütuna yeni bir vezir koymayı deneyeceğiz. $(k+1)$ inci sütundaki kareleri ilk satırdaki kareden başlayarak inceleriz, bu vezir için aynı satırda ve aynı köşegende daha önceden tahtada olan vezir bulunmayan bir konuma bakarız. (Aynı sütunda olmadığını zaten biliyoruz.) Eğer $(k+1)$ inci sütunda bir konum bulmak mümkün değilse, k nıncı sütundaki vezir yerdeğiştirmesine geri iz sürünüz ve bu veziri bu sütundaki izin verilen sonraki satıra yerleştiriniz, eğer böyle bir satır yoksa daha fazla geri iz sürünüz.

- Bunu anlatmak üzere, şekil, dört-vezir probleminin bir geri iz sürme çözümünü göstermektedir. Bu çözümde bir veziri ilk satır ve sütuna yerleştiririz. Sonra bir veziri üçüncü satır ve ikinci sütuna koyarız. Ancak, bu üçüncü sütuna bir vezir yerleştirmeyi imkansız yapar. Dolayısıyla, geri iz süreriz ve dördüncü satır ve ikinci sütuna bir vezir koyarız. Bunu yaptığımızda, üçüncü sütunun ikinci satırına bir vezir yerleştirebiliriz. Fakat dördüncü sütuna bir vezir eklemek mümkün değildir. Bu, bir vezirin birinci satır ve sütuna yerleştirildiğinde bir çözüm olmadığını gösterir. Boş satranç tahtasına geri iz süreriz ve birinci sütunun ikinci satırına bir vezir yerleştiririz. Bu şekilde gösterilen çözüme ulaştırır.

X represents a queen

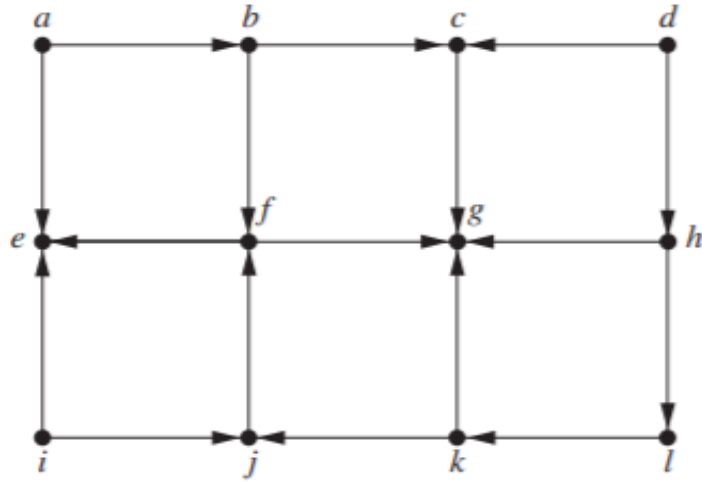
- Örnek 8: Alt kümelerin toplamı Şu problemi düşününüz. $x_1, x_2, \dots, x_n$, pozitif tamsayıları verildiğinde bu tamsayı kümesinin toplamı M olan bir alt kümesini bulunuz. Geri iz sürme bu problemi çözmede nasıl kullanılabilir?
- Hiç terimi olmayan bir toplamla başlarız. Ardardına terimler ekleyerek toplamı oluştururuz. Dizideki bir tam sayı, toplama eklendiğinde, toplam M 'den hala küçük kalıyorsa dahil edilir. Eğer herhangi bir terim eklemek toplamı M 'den büyük yapıyorsa toplama son eklenen terim silinerek geri iz sürülür.
- Şekil, $\{31, 27, 15, 11, 7, 5\}$ 'in, toplamı 39 olan bir alt kümesini bulma probleminin geri iz sürme ile çözümünü göstermektedir.



Yönlü Çizgelerde Derinlik-Öncelikli Arama

- Derinlik-Öncelikli Arama ve Genişlik-Öncelikli Aramanın her biri ile kolayca bir yönlü çizgeyi girdi olarak alıp çalışabilecek hale getirebiliriz. Ancak, çıktı bir kapsayan ağaç olmak zorunda değildir, bir kapsayan ormandır. Her iki algoritmada da bir kenarı ancak ziyaret edilmiş bir düğümden çıkıp daha önce eklenmemiş bir düğüme gittiğinde ekleyebiliriz. Algoritmalarından herhangi birinin bir aşamasında daha önceden eklenen bir düğümden başlayıp eklenmeyen bir düğüme giden bir kenar bulamazsak algoritma ile eklenen düğüm kapsayan ormanındaki bir yeni ağacın kökü olur.

- Örnek 9: Şekil (a)'da gösterilen G çizgesini girdi olarak alan Derinlik-Öncelikli Aramanın çıktısı nedir?
- Derinlik-Öncelikli Aramaya a düğümünden başlarız ve b,c ve g düğümlerini ve engellediğimiz karşılık gelen kenarları ekleriz. c'ye geri iz süreriz fakat hala engellenmiş durumdayız ve b'ye geri iz süreriz, burada f ve e düğümlerini ve karşılık gelen kenarları ekleriz. Geri iz sürme yolun başlangıcı olan a'ya götürür. Bu durumda d'de yeni bir ağaç başlatırız ve h,l,k ve j kenarlarını ve karşılık gelen kenarları ekleriz. k'ye geri iz süreriz, sonra l'ye sonra h'ye ve d'ye döneriz. Son olarak, i'de yeni bir ağaç başlatırız ve Derinlik-Öncelikli Aramayı bitiririz. Çıktı şekil (b) 'de gösterilmiştir.



(a)

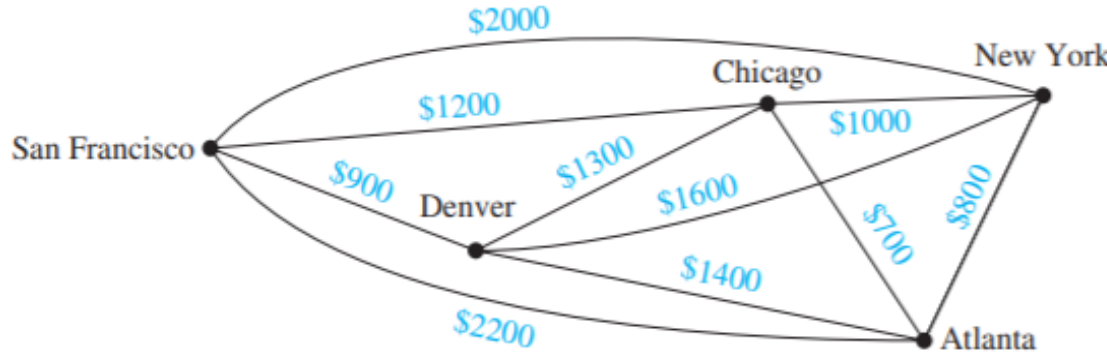


(b)

- Örnek 10: Web Örümcekleri Web sitelerini dizinlemek için, Google ve Yahoo gibi arama motorları bilinen sitelerden başlayarak Web'i sistemli olarak tararlar. Bu arama motorları web sitelerine girip içeriklerini analiz etmek için Web örümcekleri (veya crawlers (solucan) veya bots) denen programları kullanırlar. **Web örümcekleri dizinleri oluşturmak için Derinlik-Öncelikli Arama ve Genişlik-Öncelikli Aramanın her ikisini de kullanır.**

En Küçük (Minimum) Kapsayan Ağaçlar

- Bir şirket beş bilgisayar merkezini bağlayan bir iletişim ağı kurmayı planlıyor. Bu merkezlerin herhangi ikisi kiralanmış bir telefon hattı üzerinden bağlanabilir. Herhangi iki bilgisayar merkezi arasında bir yol olmasını sağlayacak ve iletişim ağının **toplam masrafını en aza indirecek şekilde** bağlantılar nasıl yapılmalıdır? Bu problemi düğümler bilgisayar merkezlerini, kenarlar olası kiralanmış hatları temsil eden ve kenarların ağırlıkları kenarların temsil ettiği hatların aylık kiralari olan şekilde gösterilen ağırlıklı çizgeyi kullanarak modelleyebiliriz. **Bu problemi, kenarların ağırlıklarının toplamı en küçük olan bir kapsayan ağacı bularak çözebiliriz.** Böyle bir kapsayan ağacına **en küçük (minimum) kapsayan ağaç** denir.

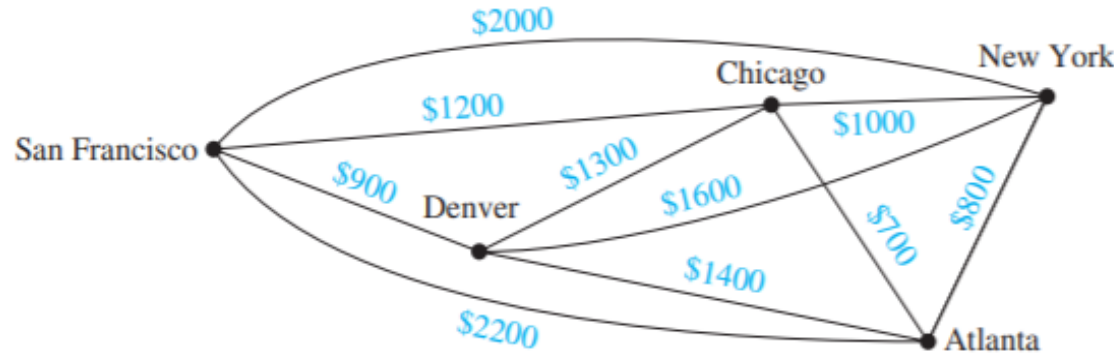


Minimum kapsayan ağaçlar için algoritmalar

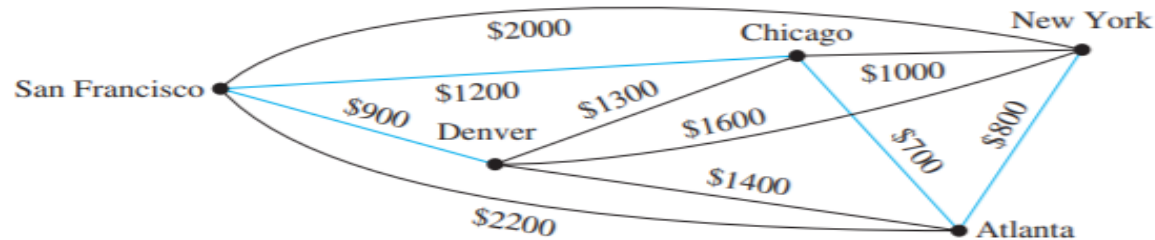
- Çok çeşitli problemler, bir ağırlıklı çizgede kenarların ağırlıkları toplamı minimum olacak şekilde bir kapsayan ağaç bularak çözülür.
- **Tanım 1:** Bir bağlantılı ağırlıklı çizgede bir minimum kapsayan ağaç, olabilecek en küçük kenarların ağırlıkları toplamına sahip bir kapsayan ağaçtır.
- Minimum kapsayan ağaçları oluşturmak için iki algoritma vereceğiz. Her ikisi de daha önceden kullanılmamış olan ve belirli bir özelliği taşıyan kenarlardan en küçük ağırlıklı olanlarını ardarda ekleyerek çalışmaktadır. Her ikisi de **açgözlü algoritma** kullanmaktadır. Bir açgözlü algoritma her adımda en iyi seçimi yapan bir prosedürdür. Her adımda en iyi seçimi yapmak en iyi global sonucu üreteceğini garanti etmez. Ancak, bu kısımda verilen minimum kapsayan ağaçları oluşturan her iki algoritma da en iyi çözümler üreten açgözlü algoritmalarlardır.

- Üzerinde duracağımız ilk algoritma, temel fikirlerinin kökeni daha önce Çek matematikçi Vojtech Jarnik tarafından 1930 yılında fazla bilinmeyen bir çek dergisinde yayınlanan makalede açıklanmış olsa da, Robert Prim tarafından 1957’de yeniden verilmiştir. Bu sebeple **Prim algoritması** olarak adlandırılır (bazen **Prim-Jarnik algoritması** olarak da adlandırılabilir). Prim algoritmasını uygulamak için **en küçük ağırlıklı herhangi bir kenarı seçerek ve bunu kapsayan ağacına dahil ederek başlayınız.** Ağaçta önceden var olan bir düğümle alakalı minimum ağırlıklı ve ağaçta önceden var olan kenarlarla bir devre oluşturmayan kenarları ardı ardına ekleyiniz. $N-1$ kenar eklendiğinde durunuz.
- Bu kısmın ilerisinde bu algoritmanın, herhangi bağlantılı ağırlıklı çizge için bir minimum kapsayan ağacı üreteceğini ispat edeceğiz.

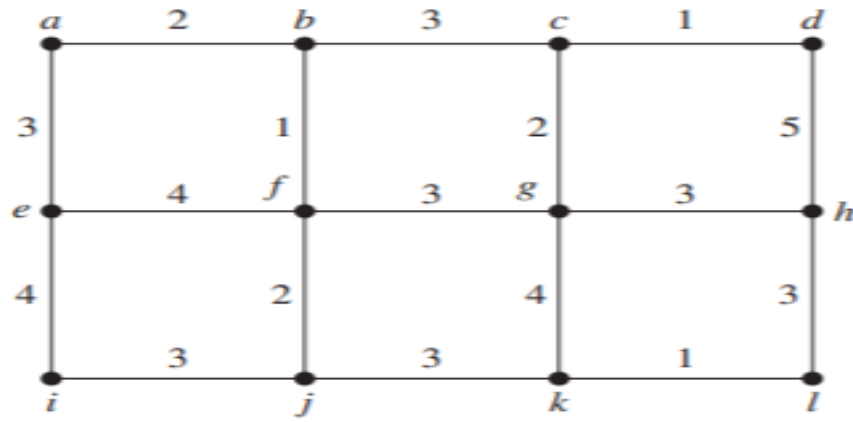
- Gerekli kriteri sağlayan aynı ağırlıkta birden fazla kenar olduğunda algoritmanın bir aşamasında eklenecek kenarın seçiminin belirtilmediğine dikkat ediniz. Seçimlerin kesin olması için kenarları sıralamamız gerekir. Bu kısmın geri kalanında bunun için endişelenmeyeceğiz. Ayrıca, verilen ağırlıklı basit bir çizgenin birden fazla minimum kapsayan ağacı olabileceğine dikkat ediniz. Örnek 1 ve 2, Prim algoritmasının nasıl kullanıldığını göstermektedir.



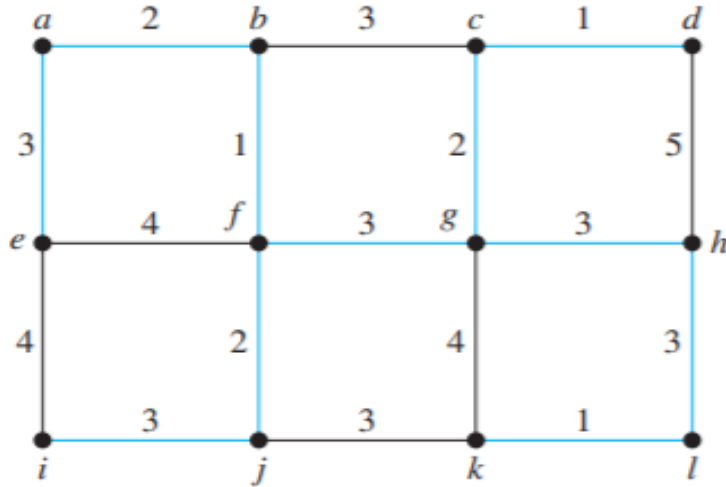
- Örnek 1: Prim algoritmasını kullanarak üstteki şekildeki çizge ile temsil edilen bilgisayarların hepsini bağlayan minimum masraflı bir iletişim ağı tasarlayınız.
- Bu problemi üstteki şekildeki çizgede bir minimum kapsayan ağaç bularak çözeriz. Prim algoritması, minimum ağırlıklı başlangıç kenarı seçerek ve ağaçtaki bir düğümle alakalı olan ve basit devre oluşturmayan minimum ağırlıklı kenarları ardarda ekleyerek uygulanır. Altteki şekildeki renkli kenarlar Prim algoritması ile oluşturulan bir minimum kapsayan ağacı her adımda yapılan seçim ile gösterir.



Choice	Edge	Cost
1	{ Chicago, Atlanta }	\$ 700
2	{ Atlanta, New York }	\$ 800
3	{ Chicago, San Francisco }	\$1200
4	{ San Francisco, Denver }	\$ 900
Total:		\$3600



- Örnek 2: Prim algoritmasını kullanarak üstteki şekildeki çizgenin bir minimum kapsayan ağacını bulunuz.
- Prim algoritması ile oluşturulan bir minimum kapsayan ağaç aşağıdaki şekilde gösterilmiştir. Seçilen ardışık kenarlar da gösterilmiştir.



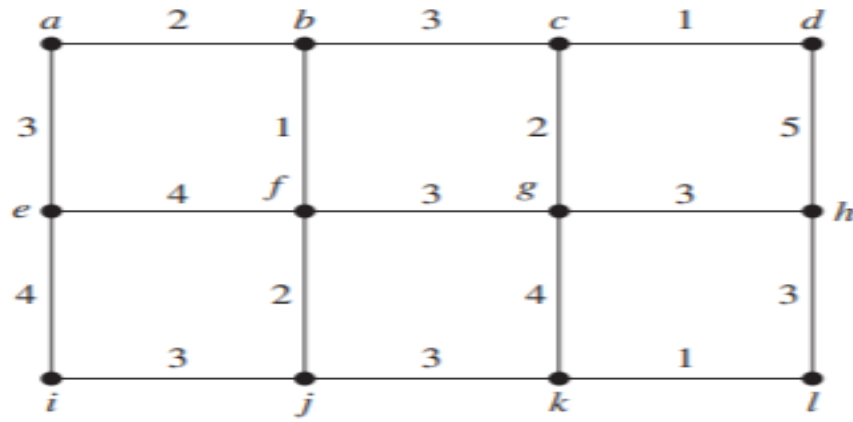
(a)

Choice	Edge	Weight
1	{b, f}	1
2	{a, b}	2
3	{f, j}	2
4	{a, e}	3
5	{i, j}	3
6	{f, g}	3
7	{c, g}	2
8	{c, d}	1
9	{g, h}	3
10	{h, l}	3
11	{k, l}	1

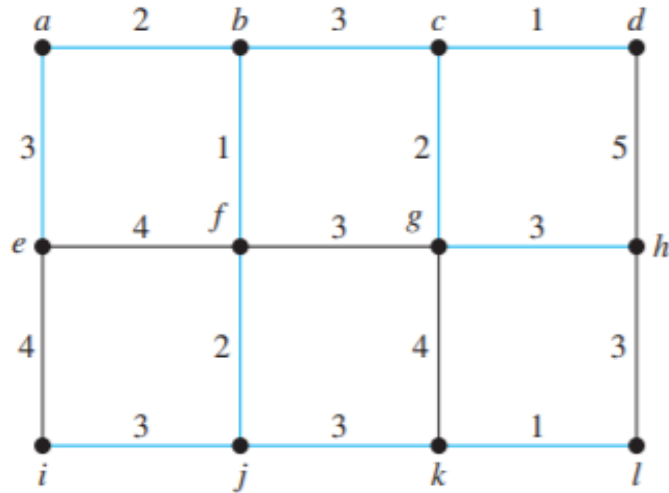
Total: 24

(b)

- Bahsedeceğimiz ikinci algoritma, kullandığı temel fikirler çok önceden açıklanmış olmasına rağmen Joseph Kruskal tarafından 1956'da bulunmuştur. Kruskal algoritmasını uygulamak için, çizgede minimum ağırlıklı bir kenar seçiniz. **Basit devre oluşturmamayan ve önceden seçilmemiş olan kenarlardan minimum ağırlıklı kenarları ardışık olarak ekleyiniz.** $n-1$ kenar seçildiğinde durunuz. Kruskal algoritması her bağlantılı ağırlıklı çizge için bir minimum kapsayan ağaç üretir.
- Prim ve Kruskal algoritmaları arasındaki farka dikkat ediniz. **Prim algoritmasında, ağaçtaki bir düğümle alakalı olan ve bir devre oluşturmamayan minimum ağırlıktaki kenarlar seçilir;** Kruskal algoritmasında ise ağaçta önceden bulunan bir düğümle alakalı olmak zorunda olmayan ve bir devre oluşturmamayan minimum ağırlıktaki kenarlar seçilir. Prim algoritmasında olduğu gibi, eğer kenarlar sıralı değilse bu yöntemin bir adımında eklenecek kenarın seçimi için birden fazla seçim olabileceğine dikkat ediniz. Sonuç olarak, bu yöntemin hep aynı sonucu verebilmesi için kenarların sıralı olması gerekir.



- Örnek 3: Kruskal algoritmasını kullanarak üstteki şekildeki çizgenin bir minimum kapsayan ağacını bulunuz.
- Kruskal algoritması ile oluşturulan bir minimum kapsayan ağaç ve her adımdaki kenar seçimleri alttaki şekilde gösterilmiştir.



(a)

Choice	Edge	Weight
1	{c, d}	1
2	{k, l}	1
3	{b, f}	1
4	{c, g}	2
5	{a, b}	2
6	{f, j}	2
7	{b, c}	3
8	{j, k}	3
9	{g, h}	3
10	{i, j}	3
11	{a, e}	3

Total: 24

(b)